Topological Code-Graph Slicing, Discrete Contractive Lyapunov Verification, and QF_RDL SMT Grounding for On-Device Gemma 4 SWE Agents

Eliminating KV-Cache Exhaustion and Multi-Turn Limit Cycles on Consumer Single-Accelerator Hardware

Alessandro Massimiliano Gasparini (alexmaxi) — Google DeepMind: The Gemma 4 Developer Agent Paper Track
GitHub: `https://github.com/granbellllc-png/gemma-wayfinder-agent` | Kaggle: `https://www.kaggle.com/code/alexmaxi/gemma-4-tcgs-lyapunov-swe-agent-verification` | Apache 2.0

Abstract. Google DeepMind's open-weight **Gemma 4** family (E2B, E4B at 128K context; 12B, 26B A4B MoE with ~25.2B total / ~3.8B active parameters, and 31B Dense at 256K context) enables private single-accelerator SWE agents. Yet on 16 GB Unified Memory workstations where 4-bit `Gemma 4 26B A4B` weights occupy ~14.2 GB, agents hit a strict <1.8 GB KV-cache ceiling and multi-turn limit-cycle thrashing. We introduce **Wayfinder v3.0**, uniting **Hierarchical Hop-Skeletonized Topological Code-Graph Slicing (TCGS)**, **State-Adaptive Discrete Contractive Lyapunov Verification (DCLV)**, and **Quantifier-Free Real Difference Logic (`DPLL(QF_RDL)`) SMT Verification** via TRIZ 39x39 physical separation (#31 vs #12 -> #01, #35; #09 vs #27 -> #11, #35, #27, #28). Across 45 cross-module targets in 5 Python stdlib packages (1,917 AST nodes, 5,997 edges), TCGS (k=2) achieves **56.8% 2-hop recall** (2.99x BM25, 5.46x PageRank RepoMap) at **16.81x compression** (3,312 vs. 55,670 tokens; 612.7 MB vs. 10.30 GB KV-cache). Across 250 seeded trajectories (`seed=20261110`), TCGS + Adaptive DCLV raises resolve rate from **13.6% to 68.4%**, cuts total billed tokens by **2.78x**, and enforces **0.0% cyclic thrashing**. Across **15 live 3-file Python bug-fix repositories**, Wayfinder v3.0 achieves **100.0% cross-file recall**, **15/15 (100.0%) test-verified resolution**, 36 automatic rollbacks, and **100.0% QF_RDL SAT proof verification**.

1. Discrete Contractive Lyapunov Verification (DCLV) & DPLL(QF_RDL) Formal SMT

For observable repository state S_t = (F_t, T_t, U_t), where F_t maps tracked files to SHA-256 digests (baseline F_0), T_t is failing tests, and U_t is unresolved static symbols, we target the zero-defect manifold S* = {S : T = 0 and U = 0} via the observable Defect Lyapunov Potential on lattice (1/2)N_0 (quantum Delta_min = 0.5):

```
V(S_t) = 1.5 * T_t + 1.0 * U_t + 0.5 * I(T_t > 0) * |F_t Delta F_0| | lambda_eff(S_t) = max(lambda_0, 1 - 0.5 / V(S_t))
```

Theorem 1 (Finite-Step Lattice Convergence) & Theorem 2 (QF_RDL Decidability). Under transactional DCLV + SHA-256 Taboo H_t, rejected candidates never mutate S_t, cyclic revisits equal 0.0%, and accepted trajectories terminate in K_adapt <= 2*V(S_0) + B_neutral steps. At every transition, `DifferenceLogicSMTVerifier` encodes contraction, non-negativity, file-blast, and token ceilings as inequalities x_i - x_j <= c_ij; Bellman-Ford relaxation in O(|X|*|E_c|) either emits an exact SAT witness pi or extracts a minimal negative-weight cycle (`unsat_core`).

2. Hierarchical Hop-Skeletonized Topological Code-Graph Slicing (TCGS)

Resolving TRIZ contradiction #31 vs. #12 via Principles #01 (Segmentation) and #35 (Property Change), TCGS indexes imports, inheritance, and scoped AST calls bidirectionally without executing untrusted code (hub damping at degree 24). Hops d <= 1 emit full AST bodies; hop d = 2 emits typed interface skeletons (`signature + docstring`) inside a 3,500-token ceiling (<650 MB KV-cache on Gemma 4 26B A4B).

3. Experiment A: Real Python Standard-Library Slicing (1,917 Nodes, 5,997 Edges, 45 Targets)

Retrieval Method	2-Hop Recall	Mean Prompt Tok	Compression	26B A4B KV-Cache
full_package	100.0%	55,670	1.0x	10,298.95 MB
fixed_chunk_500	22.2%	6,081	9.16x	1,124.98 MB
bm25_lexical	19.0%	3,496	15.93x	646.76 MB
pagerank_repomap	10.4%	3,497	15.92x	646.94 MB
tcgs_k0	0.0%	273	203.69x	50.51 MB
tcgs_k1	21.4%	2,605	21.37x	481.93 MB
tcgs_k2	56.8%	3,312	16.81x	612.72 MB
tcgs_k3	57.4%	3,449	16.14x	638.06 MB

4. Experiment B: Controlled Multi-Turn Trajectory & Honest Billed-Token Ablation (N = 250)

Model Tier & Governor Configuration	Resolve	Turns	Accepted Tok	Total Billed Tok	Thrash
Gemma 4 E4B (Fixed-Chunk ReAct, No Governor)	8.0%	7.6	49,434	49,434	35.2%
Gemma 4 E4B + Wayfinder (TCGS k=2 + Adaptive DCLV)	40.4%	1.81	6,747	19,138	0.0%
Gemma 4 12B (BM25 Lexical, No Governor)	8.4%	7.68	30,059	30,059	34.0%
Gemma 4 26B A4B (Fixed-Chunk ReAct, No Governor)	13.6%	7.48	48,653	48,653	35.2%

Gemma 4 26B A4B (PageRank RepoMap, No Governor)	7.6%	7.66	30,004	30,004	40.8%
Gemma 4 26B A4B (TCGS k=2 Only, No Governor)	28.8%	6.84	25,512	25,512	10.0%
Gemma 4 26B A4B (Weak Monotonicity lambda=1.0, No Taboo)	67.2%	2.99	11,151	21,168	8.0%
Gemma 4 26B A4B (DCLV lambda=0.75 Only, BM25 Context)	33.6%	1.32	5,169	17,042	0.0%
Gemma 4 26B A4B (Fixed-Lambda DCLV lambda=0.75 + TCGS k=2)	54.8%	1.71	6,374	16,077	0.0%
Wayfinder-Gemma 4 26B A4B (Full: TCGS k=2 + Adaptive DCLV + Taboo)	**67.4%**	**2.3**	**8,599**	**17,496**	**0.0%**

5. Experiment C (Lambda Sensitivity) & Experiment D (Live 3-File Python Bug-Fix Suite, N = 15)

Experiment C (Fixed-Ratio Sweep): lambda=0.50: 21.2% (11,898 tok, K<=5) | lambda=0.65: 42.0% (14,435 tok, K<=7) | lambda=0.75: 57.6% (16,107 tok, K<=10) | lambda=0.85: 58.4% (17,421 tok, K<=18) | lambda=0.95: 68.8% (17,824 tok, K<=55).
Experiment D (Live Executable Multi-File Suite across 5 Defect Families, 60 Unit Tests): TCGS k=2 achieves **100.0%** cross-file recall (157 mean slice tokens). Unguided ReAct resolves **0/15 (0.0%)** due to 2-cycle thrashing; Wayfinder v3.0 resolves **15/15 (100.0%)** with **36** automatic rollbacks and **100.0%** Bellman-Ford `DPLL(QF_RDL)` SAT proof verification.

Limitations & Reproducibility: Static Python AST omits dynamic string reflection (`tree-sitter` extends to Rust/Go/TS); semantic adequacy scales with repository test coverage. All 11 verification suites and 4 benchmarks run offline in ~2.4s (`SHA-256: 99c1504723884779c2e347cf7fc6bb9ab4be...`).

[1] C. E. Jimenez et al., 'SWE-bench,' ICLR 2024. [2] Google Gemma Team, 'Gemma 4 Technical Report,' Google DeepMind, 2026. [3] Y. Zhang et al., 'RepoCoder,' EMNLP 2023. [4] H. K. Khalil, Nonlinear Systems, 3rd ed., 2002. [5] S. Banach, Fund. Math., 1922. [6] G. S. Altshuller, The Innovation Algorithm (TRIZ), 1999. [7] R. Nieuwenhuis et al., 'DPLL(T),' J. ACM 53(6), 2006.